

NakedAgent: A Zero-Dependency Coding Agent Architecture via Standard-Library Primitives and Provable Replay

Reuben Bowlby

HUMMBL, LLC

reuben@hummbll.io

ORCID: [0009-0002-5620-1103](https://orcid.org/0009-0002-5620-1103)

September 2026

Abstract

Autonomous software engineering agents driven by Large Language Models (LLMs) increasingly rely on expansive, multi-tiered dependency trees. Mainstream agent scaffolds (e.g., SWE-agent, Aider, OpenHands) import between 20 and 50+ third-party packages to facilitate HTTP communication, terminal UI rendering, provider protocol abstraction, and telemetry. While feature-rich, this architectural paradigm introduces substantial supply-chain vulnerability attack surfaces, runtime version fragility, and heavy deployment footprints. In this paper, we introduce **NakedAgent**, an autonomous coding agent engineered under a strict zero-runtime-dependency constraint: it executes entirely upon standard Python 3.10+ library modules (`urllib`, `json`, `subprocess`, `re`, and `hashlib`). NakedAgent decouples tool invocation from proprietary JSON function-calling schemas by utilizing a Markdown fenced code-block protocol, enabling consistent tool use across local models (such as Qwen 2.5-Coder and Gemma 3) and hosted endpoints. To establish auditability and non-repudiation, we model agent execution as a pure Mealy state machine reducer $(S_t, E_t) \rightarrow (S_{t+1}, A_{t+1})$, combined with SHA-256 Merkle-linked state hashing recorded in an append-only JSON Lines (JSONL) event log. This architecture enables zero-inference offline replay verification, validating tool execution histories without querying an LLM. We present empirical findings from a SWE-bench Lite pilot with local 12B models, delineating scaffold-induced failure modes from model-inherent reasoning bounds. Finally, we formulate the *Omakase* extensibility doctrine, demonstrating how clean tool substitution seams empower extensible agent behaviors without framework bloat.

1 Introduction

Large Language Models have catalyzed a paradigm shift in software engineering automation [4, 9]. Modern coding agents read repositories, analyze stack traces, formulate patch hypotheses, and execute shell commands iteratively within feedback loops [10, 7].

However, existing state-of-the-art coding agent frameworks—including Aider [3], SWE-agent [9], gptme [1],

and OpenHands [8]—are characterized by substantial dependency bloat. Typical installations pull in dozens of transitive packages spanning asynchronous networking libraries (`httpx`, `aiohttp`), rich terminal user interface kits (`rich`, `prompt_toolkit`), distributed telemetry, and vendor-specific SDKs.

This dependency proliferation incurs three critical systemic liabilities:

1. **Supply-Chain Attack Surfaces:** Every additional package imported into an agent runtime introduces latent Common Vulnerabilities and Exposures (CVEs) and vectors for malicious compromise [6, 5]. Because coding agents inherently possess arbitrary shell execution and filesystem modification privileges within their execution sandbox, compromising an agent’s dependency tree yields immediate remote code execution.
2. **Version Fragility and Upstream Churn:** Minor version updates across transitive dependencies frequently break agent loops or corrupt internal prompt structures, undermining long-term production stability.
3. **Container and Footprint Overhead:** Deploying agents into resource-constrained edge systems, embedded devices, or air-gapped secure enclaves becomes prohibitive when megabytes of external dependencies must be containerized and verified.

To confront this trade-off, we propose **NakedAgent**. NakedAgent poses a fundamental research question: *Can an autonomous coding agent achieve robust, auditable, and extensible code modification loops while maintaining exactly zero runtime dependencies?*

By restricting the runtime strictly to the standard Python library (`stdlib`), NakedAgent eliminates supply-chain exposure by construction. In this paper, we describe the architectural foundations of NakedAgent, its formal Mealy state machine execution lane, its tamper-evident Merkle event logging and offline replay mechanism, its empirical performance on SWE-bench Lite, and the Omakase extensibility doctrine.

2 System Architecture

2.1 Foundational Invariants

NakedAgent is governed by three primary architectural invariants:

$$|\mathcal{D}_{\text{runtime}}| = 0 \quad (1)$$

where $\mathcal{D}_{\text{runtime}}$ represents the set of external third-party packages required to execute the agent.

$$\forall t \in \mathcal{T}_{\text{foundation}}, \quad t : (\mathcal{A}_{\text{args}}, \mathcal{W}_{\text{context}}) \rightarrow \mathcal{R}_{\text{result}} \quad (2)$$

where $\mathcal{T}_{\text{foundation}}$ is a minimal set of four deterministic tools operating within workspace $\mathcal{W}_{\text{context}}$.

2.2 The Four Foundation Tools

Rather than supplying extensive specialized APIs, NakedAgent constrains its foundation to four primitive tools:

- **read [path]**: Inspects file contents within the workspace boundary. To assist model attention across long documents, line numbering is formatted cleanly, and path escapes traversing outside the target workspace are rejected.
- **write [path]**: Atomically writes or overwrites content to disk. Directory paths are created on demand, preventing path traversal vulnerabilities.
- **patch [path]**: Applies search-and-replace hunks. Rather than relying on non-deterministic diff algorithms, the patch tool requires exact, unique substring matches:

$$\text{count}(S_{\text{search}}, F_{\text{content}}) = 1 \quad (3)$$

If the search target occurs zero times (missing) or more than once (ambiguous), the patch fails closed with an explicit diagnostic message, preventing silent code corruption.

- **shell [cmd]**: Executes system commands via `subprocess.run`. In interactive mode, shell invocations require human confirmation. In automated or headless runs, shell access is guarded by an explicit `--allow-shell` flag and a tokenized, prefix-based allowlist (`--shell-allowlist`).

2.3 Schema-Free Tool Calling via Code Fences

Most commercial agent frameworks rely on proprietary JSON function-calling formats (e.g., OpenAI Tool Calls, Anthropic Tool Use). However, open-weights and smaller localized models (e.g., 7B–14B parameter models running on consumer GPUs via Ollama) frequently struggle with strict JSON schema serialization, resulting in decoding failures and aborted trajectories.

NakedAgent resolves this via *Markdown Code-Fence Tool Parsing*. The system prompt instructs the language model to invoke tools by emitting standard triple-backtick fenced blocks where the language identifier denotes the tool:

```
```patch src/calculator.py
<<<<<< SEARCH
def add(a, b):
 return a - b
=====
def add(a, b):
 return a + b
>>>>>> REPLACE
```
```

A deterministic regular expression scanner extracts tool blocks from the generated token stream:

$$\mathcal{P}_{\text{tool}} : \text{Response} \rightarrow \{(\text{name}_i, \text{target}_i, \text{payload}_i)\}_{i=1}^k \quad (4)$$

Because every modern instruction-tuned model is pre-trained extensively on Markdown code blocks, tool-calling success rates remain high even on small 7B parameter models without fine-tuned JSON function calling capabilities.

2.4 Transport Abstraction

Network transport is implemented via standard `urllib.request`. NakedAgent natively supports two wire protocols:

1. **Ollama Local API**: Default local inference via `/api/chat`, operating with zero network exposure.
2. **OpenAI-Compatible Wire API**: Supporting vLLM, LM Studio, and remote API gateways using standard Bearer token authentication loaded from environment variables (`OPENAI_API_KEY`).

3 Provable Execution and Replay Verification

3.1 The Mealy Machine State Reducer

To provide mathematical rigor and eliminate unverified hidden state, NakedAgent models automated runs as a finite state Mealy machine:

$$\delta : (S_t, E_t) \rightarrow (S_{t+1}, [A_{t+1}]) \quad (5)$$

where S_t is the immutable agent state at turn t , E_t is an incoming event, and A_{t+1} represents outward actions dispatched to the environment (such as model requests or tool executions).

The agent state S_t is a frozen dataclass encapsulating:

$$S = \langle \text{status}, \text{messages}, \text{budget}, \text{pending_actions}, \text{meta} \rangle \quad (6)$$

Transitions are purely functional and side-effect free: all I/O is isolated in a thin driver layer (`driver.py`).

3.2 Merkle State Hashing and Event Logging

Every transition is logged sequentially into an append-only JSONL stream. Each record is cryptographically linked to the preceding state through SHA-256 Merkle chaining:

$$H_0 = \text{SHA-256}(\text{GenesisHeader}) \quad (7)$$

$$H_t = \text{SHA-256}(H_{t-1} \parallel \text{CanonicalJSON}(E_t) \parallel \text{Digest}(S_t)) \quad (8)$$

This cryptographic formulation guarantees:

- **Tamper Evidence:** Any post-hoc mutation of tool outputs, prompt injections, or agent actions invalidates all downstream hashes in the chain.
- **Strict Causality:** Events cannot be reordered or inserted retroactively.

3.3 Zero-Inference Offline Replay

Auditing AI agent trajectories historically required either re-executing expensive LLM calls (which introduces non-determinism) or trusting unverified raw text transcripts.

Under NakedAgent, any recorded event log $\mathcal{L} = [e_1, e_2, \dots, e_N]$ can be verified offline via:

```
$ python -m nakedagent.replay run.jsonl
PASS run.jsonl: verified 5 events; final hash 6
d1e09...
```

The replay verifier re-executes the pure reducer $\delta(S_t, E_t)$ against the recorded tool results without initiating any network calls or model inferences. If the reconstructed Merkle state hash matches the terminal hash in the log, the execution is mathematically certified to have followed the reducer’s operational semantics.

Epistemic Scope: Importantly, this cryptographic replay certifies **internal transcript causality and reducer determinism**—specifically, that the internal state transitions $\delta(S_t, E_t) \rightarrow S_{t+1}$ followed verified rules without offline transcript tampering. It does *not* certify external semantic correctness, external environment isolation, or model resistance to prompt injection, which must remain defended by OS-level sandboxing.

4 The Omakase Extensibility Doctrine

4.1 Defaults over Decisions

Inspired by the Japanese concept of *Omakase* (“chef’s choice”), NakedAgent provides highly tuned, opinionated defaults: out-of-the-box, a developer requires no configuration flags or parameter tuning.

4.2 The Plugin Seam

Extensibility is achieved not through heavyweight frameworks or abstract base classes, but via a filesystem convention seam:

- Plugins are plain Python files placed in `.nakedagent/plugins/` (repo-local) or `~/.nakedagent/plugins/` (user-global).
- A plugin exposes an optional `TOOLS: dict[str, ToolFunc]` and an optional `DISABLE: list[str]`.

4.3 Dynamic Prompt Adaptation

In classical agent architectures, modifying a tool requires manually modifying the system prompt text to reflect the new syntax. In NakedAgent, every tool function attaches an intrinsic `.usage` metadata attribute.

When plugins are discovered at startup, the system prompt generator dynamically renders the tool documentation block:

$$\text{Prompt}_{\text{system}} = \mathcal{B}_{\text{core}} \cup \bigcup_{t \in \mathcal{T}_{\text{active}}} t.\text{usage} \quad (9)$$

If a plugin overrides `patch` with a unified-diff implementation, its replacement `.usage` example automatically replaces the foundation documentation in the prompt without touching foundation code.

Furthermore, setting `DISABLE = ['shell', 'write']` prunes both the dispatch registry and the prompt examples, instantly yielding a provably read-only agent.

5 Sidekick Harness and Safety Tripwires

For governed environments, NakedAgent incorporates the *Sidekick Harness* and *JevNoulGate* tripwires:

- **Bounded Slot Budgets:** Every autonomous run is assigned a strict maximum turn budget B_{max} . Upon exhaustion, execution transitions immediately to a suspended state without unbounded recursion.
- **Tri-State Escalation:** Sensitive operations (such as high-impact shell commands) pass through a tripwire evaluator yielding ALLOW, DENY, or ESCALATE. In an escalate condition, the event log is sealed in a suspended state awaiting human sign-off.
- **Zero-Dependency MCP Client:** NakedAgent includes a standard-library JSON-RPC 2.0 client supporting the Model Context Protocol (MCP) over `stdio`, enabling tool sharing across modern agent meshes without external SDKs.

6 Exploratory Pilot: Failure Modes of Local Models under Minimal Scaffolding

6.1 Experimental Setup

To explore the failure dynamics, harness friction, and baseline capabilities of minimal agent architectures under local inference, we conducted an exploratory pilot on SWE-bench Lite [4].

- **Hardware:** NVIDIA GeForce RTX 3080 Ti (12 GB VRAM), Docker Desktop (WSL2), AMD Ryzen CPU.

- **Model:** Google `gemma3:12b` served locally via Ollama with 4-bit quantization. Note that while mini-swe-agent was configured with temperature $T = 0$, NakedAgent was evaluated under Ollama’s default sampling temperature (≈ 0.8), which we note as an uncontrolled exploratory variable.
- **Instances:** SWE-bench Lite instances `astropy_astropy-12907`, `-14182`, and `-14365`.
- **Baselines:** Mini-SWE-agent 2.4.6 (native tools mode vs. text-based backtick mode).

6.2 Results and Observations

Table 1 summarizes the comparative trajectories across configurations. Crucially, across all tested configurations, the true ground-truth resolution rate was **0/3 resolved (0%)**.

6.3 Error Taxonomy: Scaffold vs. Model Bounds

The pilot revealed critical insights into the interaction between minimal scaffolds and localized language models:

1. **Provider-Native Tool Incompatibility (A1):** Local Ollama serving of `gemma3:12b` rejected native function calling schemas outright. Scaffolds strictly bound to provider tool APIs failed before execution began.
2. **Syntax Brittle Failures (A2):** When forced to write raw bash blocks, small models struggled with command escaping and prompt-format compliance, resulting in `RepeatedFormatError` loops.
3. **Placeholder Echoing (C1):** In early trials, the tool usage prompt included abstract placeholders such as `<bash command>`. Gemma 3 literally echoed the angle brackets (`<git status>`), causing bash syntax errors. Replacing prompt placeholders with concrete, literal examples eliminated this error mode.
4. **Host OS Newline Ingestion (C2):** Running on Windows host systems communicating with Linux Docker containers resulted in Windows CRLF (`\r\n`) conversions over text pipes, causing git to treat entire files as modified. NakedAgent resolved this by enforcing binary streams across all tool I/O boundaries.
5. **Localized Patching (C3):** With binary I/O and concrete usage examples, NakedAgent enabled Gemma 3 to localize and generate clean single-hunk patches on complex Astropy files without external diff packages.
6. **Syntactic Compliance vs. Semantic Correctness:** Although configuration C3 generated a syntactically valid single-hunk patch on `astropy-12907`, test-suite evaluation revealed 7 unit test regressions, resulting in 0/3 resolved tasks. Minimal scaffolding effectively removes harness friction, but semantic problem-solving on complex issues remains bounded by local model parameters.

7 Security Analysis

7.1 Attack Surface Elimination

By enforcing $|\mathcal{D}_{\text{runtime}}| = 0$, NakedAgent eliminates the entire attack surface associated with dependency confusion, typosquatting, compromised sub-dependencies, and unpatched upstream CVEs in network libraries.

7.2 Shell Sandboxing and Injection Defense

Because the shell tool is the most sensitive primitive, NakedAgent implements multi-stage defensive gates:

- In automated execution, commands must match an approved argv prefix.
- Arguments are tokenized using POSIX shell lexical parsing (`shlex.split`), preventing command-chaining injection attacks (such as semicolons `;`, logical operators `&&`, or command substitutions `$ ( . . . )`).
- Path validation routines guarantee that file writes cannot escape designated workspace roots via directory traversal (`../..`).

7.3 Plugin Gating and Protected Surfaces

Eliminating third-party dependencies does not eliminate environmental risks from untrusted repositories or ambient shell binaries. To prevent repository-drop code execution (where a cloned project includes malicious code in `.nakedagent/plugins/`), NakedAgent disables workspace-local plugins by default, requiring an explicit `--trust-plugins` CLI flag. Furthermore, the `write` and `patch` tools enforce protected surface boundaries preventing modifications to `.git/` or `.nakedagent/`, and automated production workloads are strictly recommended to execute within containerized sandboxes.

8 Related Work

Coding Agents: SWE-agent [9] introduced specialized agent-computer interfaces (ACI) tailored for repository navigation. Aider [3] popularized in-terminal pair programming with git-anchored search/replace editing. OpenHands [8] established a generalist agent runtime platform. NakedAgent distinguishes itself by proving that these core capabilities can be achieved without the 30+ package dependencies characteristic of existing systems.

Execution Proofs and Governance: Krinea [2] established formal standards for append-only audit logs in AI governance. NakedAgent implements these principles at the code generation layer through its Merkle-linked Mealy state reducer and zero-inference replay verifier.

Table 1: SWE-bench Lite Exploratory Pilot Trajectory Results with Local Gemma 3 12B

| Configuration | Astropy-12907 | Astropy-14182 | Astropy-14365 |
|-------------------------------------|---------------------------------------|------------------------------------|------------------------------------|
| A1: mini-swe-agent (native tools) | BadRequest (No tool API) | BadRequest (No tool API) | BadRequest (No tool API) |
| A2: mini-swe-agent (text backticks) | RepeatedFormatError (9 calls) | LimitsExceeded (30 calls) | RepeatedFormatError (11 calls) |
| B: NakedAgent (bash-only parity) | Stopped after failed <code>sed</code> | Invalid <code>sed</code> regex | Found bug, explained, no edit |
| C1: NakedAgent (system mode, raw) | Invalid (harness bracket bug) | Invalid (harness bracket bug) | Invalid (harness bracket bug) |
| C2: NakedAgent (CRLF text pipe) | Patch applied (whole-file rewrite) | Patch applied (whole-file rewrite) | Patch applied (whole-file rewrite) |
| C3: NakedAgent (binary write fix) | 1-hunk patch (7 regressions) | No patch (nudges exhausted) | No patch (nudges exhausted) |

9 Discussion and Limitations

While standard-library minimalism delivers security and portability, it entails calculated trade-offs:

- **Lack of Streaming UI:** Without third-party libraries like `rich` or `blessed`, streaming token visualization in the terminal is intentionally rudimentary.
- **Network Transport:** Using `urllib.request` lacks built-in HTTP/2 multiplexing and requires manual connection pooling for high-throughput concurrency.
- **Patch Complexity:** The foundation `patch` tool relies on exact substring matching. While unambiguous, it requires models to reproduce exact surrounding context lines, whereas fuzzy diff engines tolerate minor whitespace variations.

10 Conclusion and Availability

NakedAgent demonstrates that a zero-runtime-dependency coding agent is not merely a theoretical curiosity, but a robust, auditable, and production-viable architecture. By relying exclusively on Python standard-library primitives, Markdown code-fence parsing, an Omakase plugin seam, and a Merkle-verified Mealy state reducer, NakedAgent provides an uncompromised foundation for autonomous software engineering.

10.1 Availability

The complete source code, test suite, benchmark harnesses, and replication logs are publicly available:

- **GitHub:** <https://github.com/hummb1-io/nakedagent>
- **PyPI:** `pip install nakedagent`
- **Zenodo Archive:** DOI: [10.5281/zenodo.23001340](https://doi.org/10.5281/zenodo.23001340)

References

- [1] Erik Bjäreholt. `gptme`: A personal ai assistant in your terminal. <https://github.com/ErikBjare/gptme>, 2023.
- [2] Reuben Bowlby. Append-only as proof: A formal account of governance sovereignty in ai audit systems. *Zenodo Preprint*, 2026.
- [3] Paul Gauthier. Aider: Ai pair programming in your terminal. <https://github.com/paul-gauthier/aider>, 2023.
- [4] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *International Conference on Learning Representations (ICLR)*, 2024.
- [5] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Jean-Rémy Falleri. Risk taxonomy for open source software supply chains. *IEEE Transactions on Software Engineering*, 2023.
- [6] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber’s knife collection: A review of open source software supply chain attacks. *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2020.
- [7] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] Xingyao Wang, Boxuan Li, Yufan Song, Frank Frank, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [9] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- [10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *International Conference on Learning Representations (ICLR)*, 2023.